

Capítulo 7

Códigos Binários

Códigos binários são esquemas especiais de representação em binário. Eles servem diversos propósitos. Note que um código binário nada mais é que uma sequência finita de bits aos quais se atribui por convenção algum significado.

Nos capítulos anteriores já foram apresentados alguns tipos de códigos binários. Como exemplo destaca-se o código de complemento de 2 utilizado para a codificação de números inteiros. Neste capítulo serão apresentados diversos códigos adicionais. Será detalhado sua regra de formação e principais usos.

7.1 Código Excesso de 3

O primeiro código binário de relevância refere-se aos códigos de excesso de N . Ele era utilizado em computadores e calculadoras antigas. A regra de formação desta família de códigos dita que uma quantidade qualquer em binário será representada pela adição de sua quantidade acrescida de N . Uma das realizações mais simples deste código refere-se ao excesso de 3. Também conhecido como código de Stibitz pode ser observado na tabela 7.1. A representação $ABCD$ do código de excesso de 3 é obtida pela soma da constante ou fator de excesso a representação N em binário.

Em sua representação mais comum aplica-se o excesso apenas às dez quantidades unitárias da base decimal, ou seja, para os números na faixa $[0, 9]$. No entanto, não há nada que explicitamente impeça que a regra se estenda para quantidades que superem o 9. Um fato interessante acerca do código de excesso de 3 refere-se que a representação em binário da quantidade 0, ou seja 0011, é o exato inverso da quantidade 9, 1100, o 1 = 0100 é o inverso de 8 = 1011 e assim sucessivamente. Outro ponto interessante é que este código não admite representações 0000 ou 1111 é que é relevante para

Decimal	Binário	A	B	C	D
0	0000	0	0	1	1
1	0001	0	1	0	0
2	0010	0	1	0	1
3	0011	0	1	1	0
4	0100	0	1	1	1
5	0101	1	0	0	0
6	0110	1	0	0	1
7	0111	1	0	1	0
8	1000	1	0	1	1
9	1001	1	1	0	0

Tabela 7.1: Tabela de correspondência entre decimal, binário em $\mathbb{N}$ e código de excesso de 3.

comunicação de dados.

7.2 Código BCD 8421

O código BCD (do inglês *Binary Coded Decimal*) é um código binário que utiliza quatro bits para representar em binário os dígitos decimais. Os números 8421 presentes no código servem para indicar o peso que os bits possuem na quantidade.

Decimal	A	B	C	D
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Tabela 7.2: Tabela de correspondência entre decimal e *binary coded decimal*.

A motivação do código é que a conversão de binário para decimal se torna simples e direta utilizando apenas bits para representar as 10 primeiras quantidades. Este código foi muito usado em computadores e calculadoras antigas.

Também encontra usos intermediários para apresentação de dados binários em displays de sete segmentos. Embora ele não utilize todo o espaço representacional ofertado pelo número de bits alocados¹, o código BCD8421 ganha em clareza pois com algum treino torna-se simples para seres humanos a conversão direta de BCD8421 para decimal sem a necessidade de consulta a tabelas ou qualquer tipo de cálculo.

Quantidades superiores a nove unidades são representadas utilizando conjuntos adicionais de 4 bits. A tabela 7.2 apresenta uma comparação entre o sistema decimal e o código BCD8421 para quantidades de 0 a 15.

Decimal	A	B	C	D	E	F	G	H
0					0	0	0	0
1					0	0	0	1
2					0	0	1	0
3					0	0	1	1
4					0	1	0	0
5					0	1	0	1
6					0	1	1	0
7					0	1	1	1
8					1	0	0	0
9					1	0	0	1
10	0	0	0	1	0	0	0	0
11	0	0	0	1	0	0	0	1
12	0	0	0	1	0	0	1	0
13	0	0	0	1	0	0	1	1
14	0	0	0	1	0	1	0	0
15	0	0	0	1	0	1	0	1

Tabela 7.3: Tabela de correspondência entre decimal e *binary coded decimal* estendida.

Exemplo 7.1. Considere o problema de se representar a quantidade 3427_{10} usando o código BCD8421. O processo representacional é simples, utilizam-se 4 bits para cada um dos dígitos da quantidade em decimal e procede-se com a conversão direta, dígito a dígito.

$$\begin{array}{cccc} \underbrace{}_3 & \underbrace{}_4 & \underbrace{}_3 & \underbrace{}_7 \\ 0011 & 0100 & 0010 & 0111 \end{array}$$

¹Com nibbles (4 bits) é possível representar 16 quantidades distintas. O código BCD8421 sacrifica 6 possíveis representações em detrimento da simplicidade representacional.

Embora a quantidade 3427_{10} pode ser representada utilizando apenas 11 bits diferentemente dos 16 bits usados pelo código BCD8421. No entanto o custo adicional em bits é pago pela simplicidade de representação. Note que o processo de conversão de decimal para BCD8421 e vice versa é muito similar a conversão binário-hexadecimal.

7.3 Código 9876543210

O código 9876543210 foi muito utilizado durante os primórdios da computação durante a época dos computadores construídos utilizando válvulas. Tais computadores eram construídos de tal forma uma válvula era utilizada para representação de cada dígito. Sendo assim este código prevê que cada dígito produzirá "1" em uma e apenas uma das dez saídas do código e todas as outras saídas serão zero.

Decimal	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	1
1	0	0	0	0	0	0	0	0	1	0
2	0	0	0	0	0	0	0	1	0	0
3	0	0	0	0	0	0	1	0	0	0
4	0	0	0	0	0	1	0	0	0	0
5	0	0	0	0	1	0	0	0	0	0
6	0	0	0	1	0	0	0	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	1	0	0	0	0	0	0	0	0
9	1	0	0	0	0	0	0	0	0	0

Tabela 7.4: Tabela de correspondência entre decimal e 9876543210.

7.4 Código de Johnson

O código de Johnson é classificado como um código cíclico pois ele é construído via o deslocamento de um conjunto pré-especificado de bits. Ele é muito utilizado no contador de Johnson que encontra aplicações em geração de ondas senoidais (ligado a uma malha de resistores), como divisor de frequência por $2n$ ao invés de 2^n obtido com contadores simples.

O código é inicializado com 10 bits 00000 11111 dos quais apenas os 5 bits a esquerda são considerados como saída. A cada incremento o bit mais a esquerda é removido, todos os são deslocados para a esquerda e o bit

Decimal	A	B	C	D	E
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	1	1
3	0	0	1	1	1
4	0	1	1	1	1
5	1	1	1	1	1
6	1	1	1	1	0
7	1	1	1	0	0
8	1	1	0	0	0
9	1	0	0	0	0

Tabela 7.5: Tabela de correspondência entre decimal e Johnson.

previamente removido é reintroduzido na posição mais a direita. A tabela 7.4 apresenta os 5 bits mais significativos (saída do código)

7.5 Código de Gray

O código de Gray inventado por Frank Gray é um código binário refletivo (RBC - *Reflected Binary Code*) no qual apenas um bit muda na representação de uma entre números sucessivos.

Decimal	A	B	C	D	Decimal	A	B	C	D
0	0	0	0	0	8	1	1	0	0
1	0	0	0	1	9	1	1	0	1
2	0	0	1	1	10	1	1	1	1
3	0	0	1	0	11	1	1	1	0
4	0	1	1	0	12	1	0	1	0
5	0	1	1	1	13	1	0	1	1
6	0	1	0	1	14	1	0	0	1
7	0	1	0	0	15	1	0	0	0

Tabela 7.6: Tabela de correspondência entre decimal e Código de Gray.

Este código foi originalmente criado e utilizado em divertimentos matemáticos até que foi descoberto por engenheiros que então encontraram usos práticos. O primeiro uso prático se deu em computadores eletromecânicos pois tais aparatos apresentavam alto grau de defeitos devido a chaveamento sucessivo das válvulas. Usando o código de Gray como método de codificação foi possível minimizar o número de chaveamentos.

Atualmente o código de Gray encontra usos em códigos de correção de erros sobretudo na codificação do sinal da televisão digital e também em algumas operadoras de TV a cabo. Mais recentemente o código é utilizado em algoritmos genéticos para codificar as operações de mutação genética.

A regra de formação do código de Gray é complexa. Como citado anteriormente, números imediatamente maiores ou menores que um dado número considerado devem diferir em apenas um bit. Para um código de Gray de 2 bits o código é trivial. Iniciando-se em zero (00) inverte-se o bit menos significativo e obtêm-se o um (01). Invertendo-se o bit mais significativo obtêm-se o dois (10) e por fim invertendo-se novamente o bit menos significativo obtêm-se o 3 (11). Como o código é dito reflectivo, para se obter o código G_3 deve-se refletir o código previamente gerado. A seguir vemos em a) a reflexão do código G_2 . Para a finalização do código G_3 simplesmente atribui-se aos quatro primeiros casos "0" e aos quatro casos subsequentes o "1" como apresentado em b).

a)

0	0
0	1
1	1
1	0
1	0
1	0
1	1
0	1
0	0

b)

0	0	0
0	0	1
0	1	1
0	1	0
1	1	0
1	1	1
1	0	1
1	0	0

A conversão de **binário para Gray** não é trivial. No entanto a expressão Booleana que a habilita é relativamente direta. A equação 7.1 apresenta a regra de formação do sistema de conversão de binário para Gray. Os bits das palavras tanto em Gray quando em binário são dadas com o bit mais significativo a esquerda.

$$G_n = [g_n, g_{n-1}, \dots, g_1, g_0] = [b_n, b_n \oplus b_{n-1}, b_{n-1} \oplus b_{n-2}, \dots, b_1 \oplus b_0] \quad (7.1)$$

Na equação 7.1, G significa a palavra em Gray, e B a palavra em binário. Cada bit da palavra é indexado por g_i ou b_i nos casos de Gray e binário respectivamente. A variável n refere-se ao comprimento em bits da palavra. Note que uma palavra em binário de n bits também possuirá n bits em sua representação no código Gray.

Exemplo 7.2. Considere o número em binário 1011_2 . Para convertê-lo para

código Gray basta que apliquemos a equação 7.1. No caso o comprimento da palavra n será igual a 4 pois a palavra é composta por quatro bits.

$$\begin{aligned} g_3 &= b_3 &= g_3 &= 1 \\ g_2 &= b_3 \oplus b_2 &= g_2 &= 1 \oplus 0 = 1 \\ g_1 &= b_2 \oplus b_1 &= g_1 &= 0 \oplus 1 = 1 \\ g_0 &= b_1 \oplus b_0 &= g_0 &= 1 \oplus 1 = 0 \end{aligned}$$

Resultando no código 1110_G .

A mesma regra se aplica a palavras de comprimento arbitrariamente grandes podem ser processadas.

Exemplo 7.3. Considere o número binário $1000\ 0011_2$. Seu comprimento será de 8 bits e o processo de conversão segue:

$$\begin{aligned} g_7 &= b_7 &= g_7 &= 1 \\ g_6 &= b_7 \oplus b_6 &= g_6 &= 1 \oplus 0 = 1 \\ g_5 &= b_6 \oplus b_5 &= g_5 &= 0 \oplus 0 = 0 \\ g_4 &= b_5 \oplus b_4 &= g_4 &= 0 \oplus 0 = 0 \\ g_3 &= b_4 \oplus b_3 &= g_3 &= 0 \oplus 0 = 0 \\ g_2 &= b_3 \oplus b_2 &= g_2 &= 0 \oplus 0 = 0 \\ g_1 &= b_2 \oplus b_1 &= g_1 &= 0 \oplus 1 = 1 \\ g_0 &= b_1 \oplus b_0 &= g_0 &= 1 \oplus 1 = 0 \end{aligned}$$

Resultando no código $1100\ 0010_G$.

A conversão de **Gray para binário** também não é trivial. No entanto a expressão Booleana que a habilita é relativamente direta. A equação 7.2 apresenta a regra de formação do sistema de conversão de Gray para binário. Os bits das palavras tanto em Gray quando em binário são dadas com o bit mais significativo a esquerda.

$$B_n = [b_n, b_{n-1}, \dots, b_1, b_0] = [g_n, b_n \oplus g_{n-1}, b_{n-1} \oplus g_{n-2}, \dots, b_1 \oplus g_0] \quad (7.2)$$

Exemplo 7.4. Considere o número em binário 1110_G . Para convertê-lo para código Gray basta que apliquemos a equação 7.2. No caso o comprimento da palavra n será igual a 4 pois a palavra é composta por quatro bits.

$$\begin{aligned} b_3 &= g_3 &= b_3 &= 1 \\ b_2 &= b_3 \oplus g_2 &= b_2 &= 1 \oplus 1 = 0 \\ b_1 &= b_2 \oplus g_1 &= b_1 &= 0 \oplus 1 = 1 \\ b_0 &= b_1 \oplus g_0 &= b_0 &= 1 \oplus 0 = 1 \end{aligned}$$

Resultando no código 1011_2 .

A mesma regra se aplica a palavras de comprimento arbitrariamente grandes podem ser processadas.

Exemplo 7.5. Considere o número binário $1100\ 0010_G$. Seu comprimento será de 8 bits e o processo de conversão segue:

$$\begin{array}{ll} b_7 = g_7 & = b_7 = 1 \\ b_6 = b_7 \oplus g_6 & = b_6 = 1 \oplus 1 = 0 \\ b_5 = b_6 \oplus g_5 & = b_5 = 0 \oplus 0 = 0 \\ b_4 = b_5 \oplus g_4 & = b_4 = 0 \oplus 0 = 0 \\ b_3 = b_4 \oplus g_3 & = b_3 = 0 \oplus 0 = 0 \\ b_2 = b_3 \oplus g_2 & = b_2 = 0 \oplus 0 = 0 \\ b_1 = b_2 \oplus g_1 & = b_1 = 0 \oplus 1 = 1 \\ b_0 = b_1 \oplus g_0 & = b_0 = 1 \oplus 0 = 1 \end{array}$$

Resultando no código $1000\ 0011_2$.

7.6 Código ASCII

O código ASCII do inglês *American Standard Code for Information Interchange* - Código Padrão Americano para o Intercâmbio de Informação - foi criado e mantido a partir de 1960 e é utilizado como código para representação de textos básico em computadores, equipamentos de comunicação, etc. O código prevê 33 códigos de controle, usados para marcação de texto (ENTER, DEL, etc) localizados em $\{[0, 31], 127\}$ e 95 códigos imprimíveis ('a', 'b', 'D', '!' etc) localizados na faixa $[32, 127]$. Note que o código ASCII utiliza apenas 128 símbolos distintos (controle + gráficos) o que requer exatamente 7 bits para sua codificação. O bit extra de uma palavra de 8 bits usualmente utilizado para representar um código em ASCII encontra vários usos. O primeiro deles refere-se a utilizar o bit extra para computar a paridade do código. Esta estratégia foi muito utilizada em sistemas de comunicação de baixa velocidade. O bit também foi utilizado para expandir o número de caracteres representáveis. A empresa Microsoft o utilizou para gerar uma versão estendida do ASCII no que ficou conhecido como *WCP - Windows Code Page*. Além da extensão da tabela ASCII a Microsoft criou diversas outras *Code Pages* que estiveram em uso até meados de 1990 quando foram substituídas pelo código UNICODE. No entanto suporte as WCPs ainda é provido pelo sistema operacional e aplicações base da empresa.

Devido ao espaço representacional reduzido, apenas 128 códigos distintos, são codificados pela tabela ASCII apenas os 33 sinais de controle alguns dos

quais nem são mais utilizados, as letras do alfabeto tanto maiúsculas quanto minúsculas, códigos para os dígitos decimais e sinais de pontuação. Tais símbolos gráficos são codificados sequencialmente o que facilita a utilização da tabela.

Símbolos	Faixa de valores(Decimal)
['0', '9']	[48, 57]
['A', 'Z']	[65, 90]
['a', 'z']	[97, 122]

Tabela 7.7: Faixas de valores notáveis para os caracteres alfanuméricos previstos pelo código ASCII.

Exemplo 7.6. Deseja-se transmitir a mensagem "Gol do Verdão" representada em ASCII de um computador A para outro B. Quais seriam as cadeias de caracteres a serem transmitidas utilizando-se a paridade par? Cada um dos caracteres deve ser codificado individualmente utilizando a tabela ASCII de 7 bits e então transmitidos. A tabela 7.6 exemplifica o processo de codificação.

Caractere	Cod. ASCII	ASCII com p. par
'G'	100 0111	0100 0111
'o'	110 1111	0110 1111
'l'	110 1100	0110 1100
' '	010 0000	1010 0000
'd'	110 0100	1110 0100
'o'	110 1111	0110 1111
' '	010 0000	1010 0000
'V'	101 0110	0101 0110
'e'	110 0101	0110 0101
'r'	111 0010	0111 0010
'd'	110 0100	1110 0100
'a'	110 0001	1110 0001
'o'	110 1111	0110 1111

Tabela 7.8:

Exercícios

1. Consulte uma tabela ASCII e proceda com a conversão para binário das seguintes sequências de caracteres:

- | | |
|---|------------------------|
| a) 42_{ascII} | b) SD_{ascII} |
| c) NO_{ascII} | d) $Digital_{ascII}$ |
| e) no_{ascII} | f) $Sistemas_{ascII}$ |
| g) $abcdefghijklmnopqrstuvwxyz_{ascII}$ | h) $OlaMundo!_{ascII}$ |

2. O que significa “BCD” no código BCD8421?
3. Converta os seguintes números em BCD8421 para binário e decimal:

BCD8421	Binário	Decimal
0100 0010 ₂		
0001 0000 ₂		
0011 0100 ₂		
1001 1001 ₂		
1000 0111 ₂		

4. Qual o maior número em decimal representável por um número em BCD8421 de 16 bits?
5. Converta os seguintes números em código de Johnson para binário e para decimal:

Johnson	Binário	Decimal
00001 11110 _J		
11111 00000 _J		
10000 01111 _J		
00111 11000 _J		
11100 00011 _J		

6. Converta os seguintes números em código de Excesso de 3 para binário e para decimal:

Excesso de 3	Binário	Decimal
0101 _{ex3}		
0100 _{ex3}		
0011 _{ex3}		
0100 _{ex3}		
0110 _{ex3}		

7. Converta os seguintes números em binário para o código de Gray:

Binário	Gray
0111 ₂	
0011 ₂	
1100 0011 ₂	
1011 0100 1000 ₂	
1001 0110 0000 ₂	

8. Converta os seguintes números em Gray para binário:

Gray	Binário
0111 _G	
0011 _G	
1100 0011 _G	
1011 0100 1000 _G	
1001 0110 0000 _G	

